

组合与继承 (OOP)

黄民烈

aihuang@tsinghua.edu.cn

<https://coai.cs.tsinghua.edu.cn/html>

课程团队：刘知远 任炬 黄民烈

上期要点回顾

- 组合与继承
- 成员访问权限

成员访问权限

继承表		继承方法					
		public		private		protected	
基类中 成员类型	public	YES	pub/yes	YES	prv/no	YES	pro/no
	private	NO	prv/no	NO	prv/no	NO	prv/no
		YES	pro/no	YES	prv/no	YES	pro/no

派生类成员函数是否能访问基类成员

基类成员在派生类中的成员类型，
派生类对象是否能访问基类成员

prv: private
pro: protected
pub: public

类似集合交运算（成员类型与继承方法之间的交）

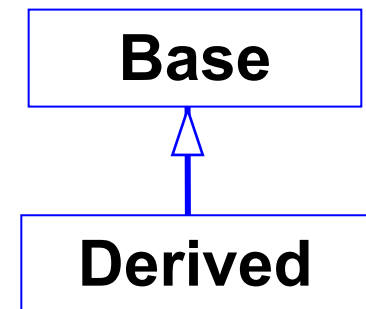
Order: public $\supset$ protected $\supset$ private

本讲内容提要

- 向上类型转换
- 对象切片
- 函数调用捆绑
- 虚函数和虚函数表
- 虚函数和构造函数、析构函数
- 重写覆盖，**override**和**final**

向上类型转换

- 派生类对象/引用/指针转换成基类对象/引用/指针，称为**向上类型转换**。只对**public**继承有效，在继承图上是上升的；对**private**、**protected**继承无效。
- 向上类型转换（派生类到基类）可以由编译器**自动完成**，是一种**隐式**类型转换。
- **凡是**接受基类对象/引用/指针的地方（如函数参数），**都****可以**使用派生类对象/引用/指针，编译器会自动将派生类对象转换为基类对象以便使用。



对象的向上类型转换

```
#include <iostream>
using namespace std;
```

```
class Base {
public:
    void print() { cout << "Base::print()" << endl; }
};
```

```
class Derive : public Base {
public:
    void print() { cout << "Derive::print()" << endl; }
};
```

```
void fun(Base obj) { obj.print(); }
```

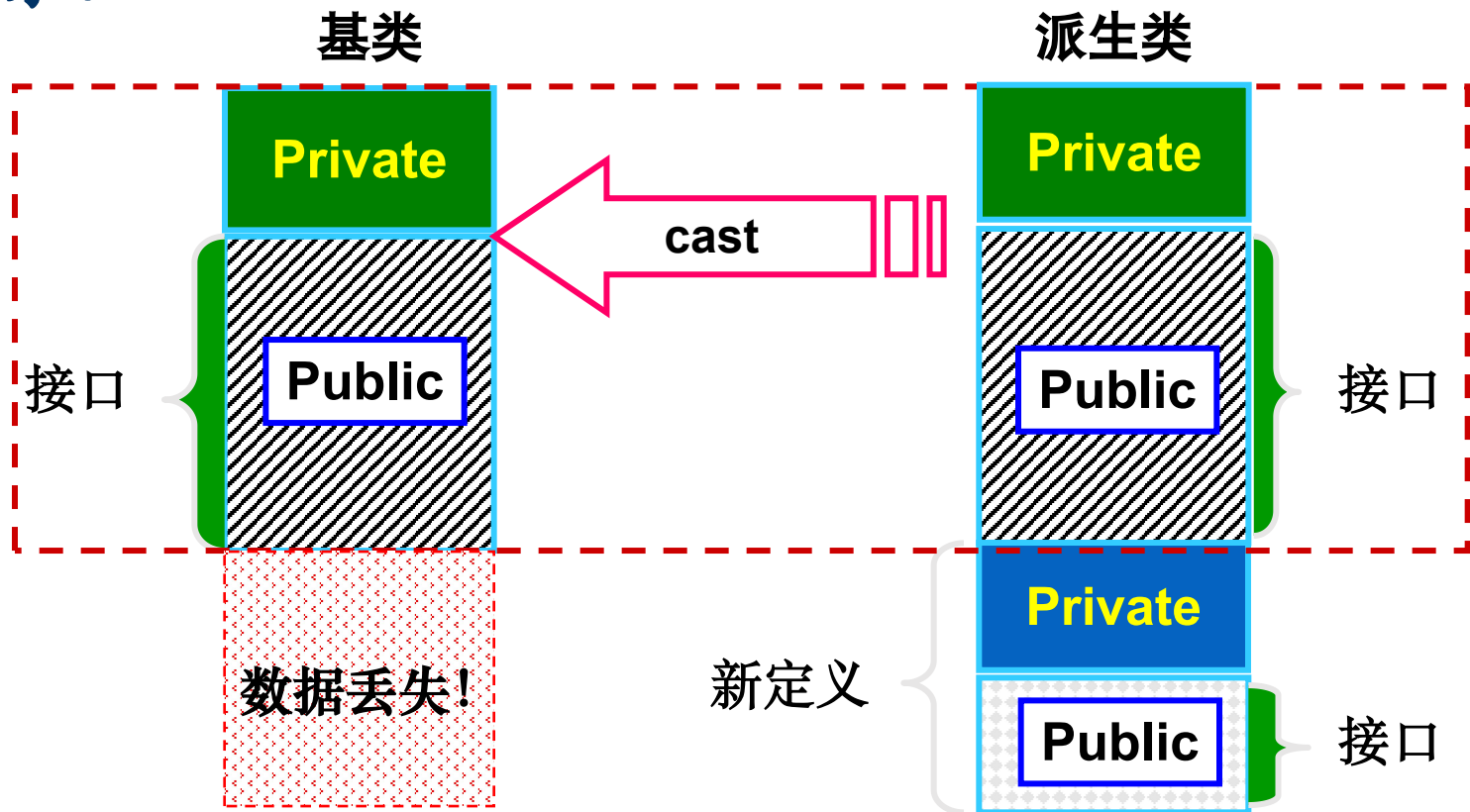
```
int main()
{
    Derive d;
    d.print();
    fun(d);    /// 本意：希望对Derive::print的调用
    return 0;
}
```

运行结果

```
Derive::print()
Base::print()
```

对象切片

- 当派生类的**对象**(不是指针或引用)被转换为基类的对象时，派生类的对象被**切片**为对应基类的子对象。



派生类新数据丢失示例

```
#include <iostream>
using namespace std;
#pragma pack(4)
class Pet {
    public: int att_i; //表示属性
    Pet(int x=0): att_i(x) {};
};
class Dog: public Pet {
    public: int att_j;
    Dog(int x=0, int y=0): Pet(x), att_j(y) {}
};
void getSize(Pet p){
    cout << "Pet size:" << sizeof(p) << endl;
}
int main() {
    Pet p;
    cout << "Pet size:" << sizeof(p) << endl;
    Dog g;
    cout << "Dog size:" << sizeof(g) << endl;
    getSize(g); /// 对象切片(传参), 数据丢失
    p = g;      /// 对象切片(赋值), 数据丢失
    cout << "Pet size:" << sizeof(p) << endl;
    return 0;
}
```

运行结果

```
Pet size:4
Dog size:8
Pet size:4
Pet size:4
```

```
#include <iostream>
using namespace std;
#pragma pack(4)
```

派生类新数据丢失示例

```
class Pet {
public: int att_i; //表示属性
    Pet(int x=0): att_i(x) {}
};
class Dog: public Pet {
public: int att_j;
    Dog(int x=0, int y=0): Pet(x), att_j(y) {}
};
int main() {
    Pet p(1);
    cout << p.att_i << endl;
    Dog g(2,3);
    cout << g.att_i << " " << g.att_j << endl;
    p = g;    /// 对象切片，只赋值基类数据
    cout << p.att_i << endl;
    //cout << p.att_j << endl; // 没有该参数，编译错误
    return 0;
}
```

运行结果

```
1
2 3
2
```

派生类新方法丢失示例

```
#include <iostream>
using namespace std;
```

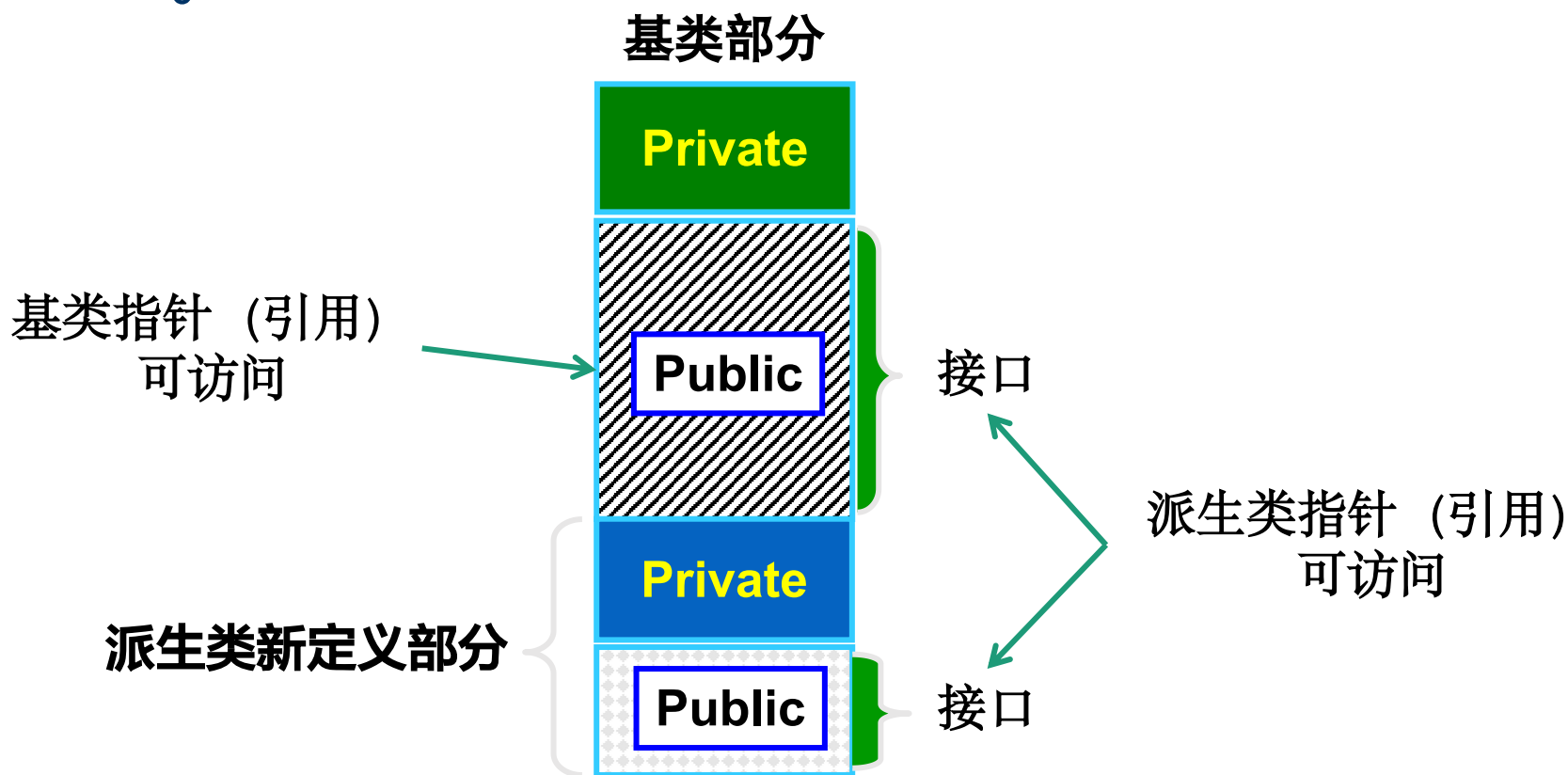
```
class Pet {
public:
    void name(){ cout << "Pet::name()" << endl; }
};
class Dog: public Pet {
public:
    void name(){ cout << "Dog::name()" << endl; }
};
void getName(Pet p){
    p.name();
}
int main() {
    Dog g;
    g.name();
    getName(g);    /// 对象切片（传参），调用基类的 name 函数
    Pet p = g;
    p.name();    /// 对象切片（赋值），调用基类的 name 函数
    return 0;
}
```

运行结果

```
Dog::name()
Pet::name()
Pet::name()
```

指针（引用）的向上转换

- 当派生类的指针（引用）被转换为基类指针（引用）时，不会创建新的对象，但只保留基类的接口。



引用的向上类型转换

```
#include <iostream>
using namespace std;
#pragma pack(4)
```

```
class Pet {
public: int att_i;
    Pet(int x=0): att_i(x) {}
};
class Dog: public Pet {
public: int att_j;
    Dog(int x=0, int y=0): Pet(x), att_j(y) {}
};
int main() {
    Dog g(2,3);
    cout << g.att_i << " " << g.att_j << endl;
    Pet& p = g;          /// 引用向上转换
    cout << p.att_i << endl;
    p.att_i = 1;         /// 修改基类存在的数据
    cout << p.att_i << endl;
    cout << g.att_i << " " << g.att_j << endl; /// 影响派生类
    return 0;
}
```

运行结果

```
2 3
2
1
1 3
```

```
#include <iostream>
using namespace std;
```

引用的向上类型转换

```
class Instrument {
public:
    void play() { cout << "Instrument::play" << endl; }
};
class Wind : public Instrument {
public:
    // Redefine interface function:
    void play() { cout << "Wind::play" << endl; }
};
```

```
void tune(Instrument& i) {
    i.play();
}
```

```
int main() {
    Wind flute;
    tune(flute); /// 引用的向上类型转换(传参), 编译器早绑定, 无对象切片产生
    Instrument &inst = flute; /// 引用的向上类型转换(赋值)
    inst.play();
    return 0;
}
```

如果**tune**的参数修改为指针?

运行结果

```
Instrument::play
Instrument::play
```

私有继承 “照此实现”

```
#include <iostream>
using namespace std;
class B {
private:
    int data{0}; ///c++11标准
public:
    int getData(){ return data;}
    void setData(int i){ data=i;}
};
class D1 : private B {
public:
    using B::getData;
};
int main() {
    D1 d1;
    cout<<d1.getData();
    //d1.setData(10);    ///隐藏了基类的setData函数，不可访问
    //B& b = d1;         ///不允许私有继承的向上转换
    //b.setData(10);     ///否则可以绕过D1，调用基类的setData函数
    return 0;
}
```

函数调用捆绑

- 把函数体与函数调用相联系称为**捆绑(binding)**。
 - 即将函数体的具体实现代码，与调用的函数名绑定。执行到调用代码时直接进入捆绑好的函数体内部。
- 当捆绑在程序运行之前（由编译器和连接器）完成时，称为**早捆绑(early binding)**。
 - 运行之前已经决定了函数调用代码到底进入哪个函数。
 - 上面程序中的问题是早捆绑引起的，编译器将tune中的函数调用i.play()与Instrument::play()绑定。
- 当捆绑根据对象的实际类型(上例中即子类Wind而非Instrument)，发生在程序运行时，称为**晚捆绑(late binding)**，又称动态捆绑或运行时捆绑。
 - 要求在运行时能确定对象的实际类型(思考：如何确定？)，并绑定正确的函数。
 - 晚捆绑只对类中的**虚函数**起作用，使用 **virtual** 关键字声明虚函数。

虚函数

- 对于被派生类重新定义的成员函数，若它在基类中被声明为虚函数（如下所示），则通过基类指针或引用调用该成员函数时，编译器将根据所指（或引用）对象的实际类型决定是调用基类中的函数，还是调用派生类重写的函数。

```
class Base {  
public:  
    virtual ReturnType FuncName(argument); //虚函数  
    ...  
};
```

- 若某成员函数在基类中声明为虚函数，当派生类重写覆盖它时（同名，同参数函数），无论是否声明为虚函数，该成员函数都仍然是虚函数。

重写覆盖虚函数

```
#include <iostream>
using namespace std;
```

```
class Instrument {
public:
    virtual void play() { cout << "Instrument::play" << endl; }
};
class Wind : public Instrument {
public:
    void play() { cout << "Wind::play" << endl; }
    /// 重写覆盖(稍后: 重写隐藏和重写覆盖的区别)
};
```

```
void tune(Instrument& ins) {
    ins.play(); /// 由于 Instrument::play 是虚函数, 编译时不再直接绑定, 运行时
    根据 ins 的实际类型调用。
}
int main() {
    Wind flute;
    tune(flute); /// 向上类型转换
    return 0;
}
```

运行结果

Wind::play

晚绑定只对 指针和引用有效

```
#include <iostream>
using namespace std;
```

```
class Instrument {
public:
    virtual void play() { cout << "Instrument::play" << endl; }
};
```

```
class Wind : public Instrument {
public:
    void play() { cout << "Wind::play" << endl; }
};
```

```
void tune(Instrument ins) {
    ins.play(); /// 晚绑定只对指针和引用有效，这里早绑定 Instrument::play
}
```

```
int main() {
    Wind flute;
    tune(flute); /// 向上类型转换，对象切片
    return 0;
}
```

运行结果

Instrument::play

虚函数表

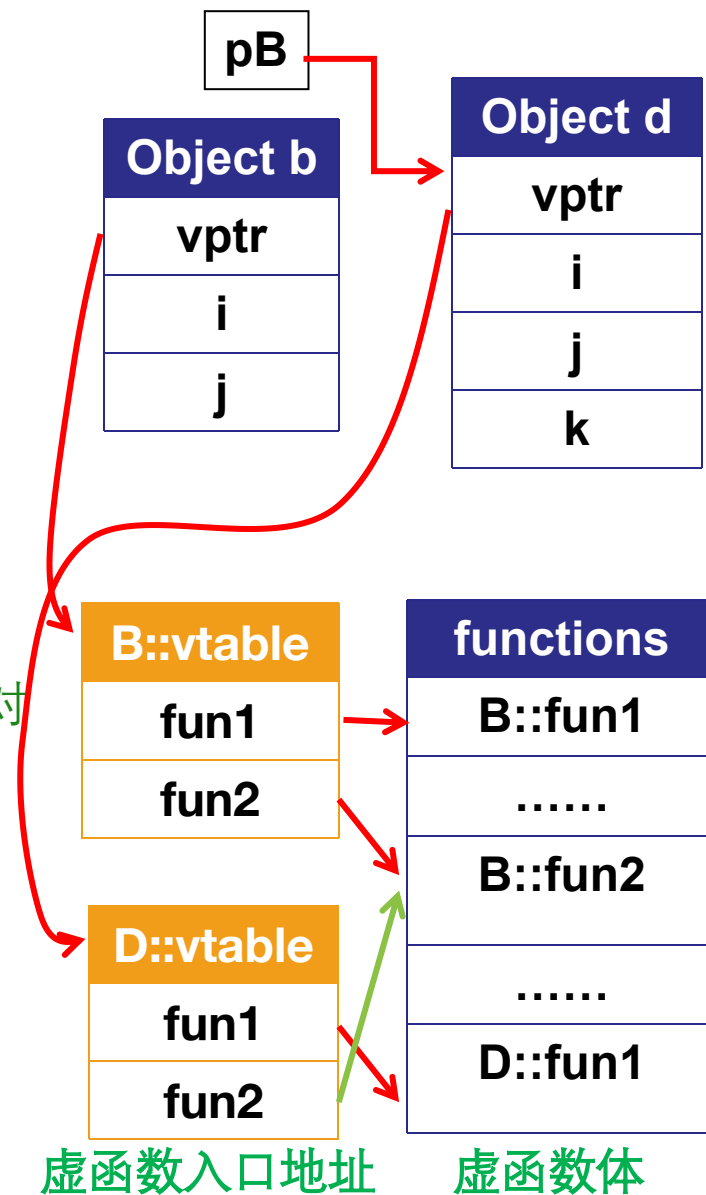
- 对象自身要包含自己实际类型的信息：用虚函数表表示。运行时通过虚函数表确定对象的实际类型。
- 虚函数表(VTABLE)：每个**包含虚函数的类**用于存储**虚函数地址**的表(虚函数表有唯一性，即使没有重写虚函数)。
- 每个包含虚函数的类**对象**中，编译器秘密地放一个指针，称为虚函数指针(vpointer/VPTR)，指向这个类的VTABLE。
- 当通过基类指针做虚函数调用时，编译器静态地插入能取得这个VPTR并在VTABLE表中查找函数地址的代码，这样就能调用正确的函数并引起晚捆绑的发生。
 - 编译期间：建立虚函数表VTABLE，记录每个类或该类的基类中所有已声明的虚函数入口地址。
 - 运行期间：建立虚函数指针VPTR，在构造函数中发生，指向相应的VTABLE。

示例

```
#include <iostream>
using namespace std;
class B{
    int i;
    float j;
public:
    virtual void fun1() {
        cout << "B::fun1()" << endl; }
    virtual void fun2() {
        cout << "B::fun2()" << endl; }
};
class D: public B{
public:
    double k;
    virtual void fun1() {
        cout << "D::fun1()" << endl; } //对fun1重写覆盖, 对
fun2没有, 则fun2使用基类的虚函数地址
};
int main() {
    B b; D d;
    B *pB = &d;
    pB->fun1();
    return 0;
}
```

运行结果

D::fun1()



存放类型信息

```
#include <iostream>
using namespace std;
#pragma pack(4) //按照4字节进行内存对齐
```

```
class NoVirtual{ //没有虚函数
```

```
    int a;
public:
    void f1() const {}
    int f2() const {return 1;}
};
```

```
class OneVirtual{ //一个虚函数
```

```
    int a;
public:
    virtual void f1() const {}
    int f2() const {return 1;}
};
```

```
class TwoVirtual{//两个虚函数
```

```
    int a;
public:
    virtual void f1() const {}
    virtual int f2() const {return 1;}
};
```

存放类型信息

```
int main(){
    cout<<"int: "<<sizeof(int)<<endl;
    cout<<"NoVirtual: "<<sizeof(NoVirtual)<<endl;
    cout<<"void* : "<<sizeof(void*)<<endl;
    cout<<"OneVirtual: "<<sizeof(OneVirtual)<<endl;
    cout<<"TwoVirtual: "<<sizeof(TwoVirtual)<<endl;
    return 0;
}
```

- 对不带虚函数的类**NoVirtual**,对象的大小就是单个**int**的大小。
- 对带有单个虚函数的类**OneVirtual**,对象的大小是单个**int**的大小加上一个**void**指针(实际上是**VPTR**)的大小。
- 带有多个虚函数的类**TwoVirtual**与**OneVirtual**大小相同,因为**VPTR**指向一个存放所有虚函数地址的表。

64位机器上运行结果

```
int: 4
NoVirtual: 4
void* : 8
OneVirtual: 12
TwoVirtual: 12
```

虚函数和构造函数、析构函数

■ 虚函数与构造函数

- 当创建一个包含有虚函数的**对象**时，必须初始化它的VPTR以指向相应的VTABLE。设置VPTR的工作由**构造函数**完成。编译器在构造函数的开头秘密的插入能初始化VPTR的代码。
- 构造函数**不能也不必**是虚函数。
 - **不能**：如果构造函数是虚函数，则创建对象时需要先知道VPTR，而在构造函数调用前，VPTR未初始化。
 - **不必**：构造函数的作用是提供类中成员初始化，调用时**明确指定**要创建对象的类型，没有必要是虚函数。

构造函数调用虚函数

```
#include <iostream>
using namespace std;
```

```
class Base {
public:
    virtual void foo(){cout<<"Base::foo"<<endl;}
    Base(){foo();} //在构造函数中调用虚函数foo
    void bar(){foo();} ///在普通函数中调用虚函数foo
};
```

```
class Derived : public Base {
public:
    int _num;
    void foo(){cout<<"Derived::foo"<<_num<<endl;}
    Derived(int j):Base(),_num(j){}
};

int main() {
    Derived d(0);
    Base &b = d;
    b.bar();
    b.foo();
    return 0;
}
```

运行结果

```
Base::foo //构造函数中调用的是foo的“本地版本”
           为什么？（提示：基类构造时_num的状态）
Derived::foo0 //在普通函数中调用虚函数
Derived::foo0 //直接调用虚函数
```

虚函数和构造函数、析构函数

■ 虚函数与构造函数

- 在构造函数中调用一个虚函数，被调用的只是这个函数的**本地版本(即当前类的版本)**，即虚机制在构造函数中不工作。
- 派生类对象初始化顺序：(与构造函数初始化列表顺序无关)
 - ① 基类初始化
 - ② 对象成员初始化
 - ③ 构造函数体
- 原因：基类的构造函数比派生类先执行，调用基类构造函数时派生类中的数据成员还没有初始化(上例中Derive中的数据成员_num)。**如果允许**调用实际对象的虚函数(如b.foo())，则可能会用到**未初始化的**派生类成员。

虚函数和构造函数、析构函数

■ 虚函数与析构函数

- 析构函数能是虚的，且常常是虚的。虚析构函数仍需定义函数体。
- 虚析构函数的用途：当删除基类对象指针时，编译器将根据指针所指对象的实际类型，调用相应的析构函数。
- 若基类析构不是虚函数，则删除基类指针所指派生类对象时，编译器仅自动调用基类的析构函数，而不会考虑实际对象是不是基类的对象。这可能会导致内存泄漏。
- 在析构函数中调用一个虚函数，被调用的只是这个函数的本地版本，即虚机制在析构函数中不工作。为什么？

虚析构函数

```
#include <iostream>
using namespace std;
```

```
class Base1 {
public:
    ~Base1() { cout << "~Base1()\n"; }
};
```

```
class Derived1 : public Base1 {
public:
    ~Derived1() { cout << "~Derived1()\n"; }
};
```

```
class Base2 {
public:
    virtual ~Base2() { cout << "~Base2()\n"; }
};
```

```
class Derived2 : public Base2 {
public:
    ~Derived2() { cout << "~Derived2()\n"; }
};
```

虚析构函数

```
int main() {  
    Base1* bp = new Derived1;  
    delete bp; /// 只调用了基类的析构函数  
    Base2* b2p = new Derived2;  
    delete b2p; /// 派生类虚析构函数调用完后调用基类的虚析构函数  
    return 0;  
}
```

运行结果

```
~Base1()  
~Derived2()  
~Base2()
```

重要原则：
总是将基类的析构函数设置为虚析构函数

重载、重写覆盖与重写隐藏

进一步阅读《C++编程思想》15.9节

■ 重载(overload):

- 函数名必须相同，函数参数必须不同，作用域相同(同一个类，或同为全局函数)，返回值可以相同或不同。

■ 重写覆盖(override):

- 派生类重新定义基类中的虚函数，函数名必须相同，函数参数必须相同，返回值一般情况应相同。
- 派生类的虚函数表中原基类的虚函数指针会被派生类中重新定义的虚函数指针覆盖掉。

■ 重写隐藏(redefining):

- 派生类重新定义基类中的函数，函数名相同，但是参数不同或者基类的函数不是虚函数。(参数相同+虚函数->不是重写隐藏)
- 重写隐藏中虚函数表不会发生覆盖。

重写覆盖与重写隐藏

进一步阅读《C++编程思想》 15.9节

■ 重写覆盖和重写隐藏：

- 相同点：
 - 都要求派生类定义的函数与基类同名。
 - 都会屏蔽基类中的同名函数，即派生类的实例无法调用基类的同名函数。
- 不同点：
 - 重写覆盖要求基类的函数是虚函数，且函数参数相同，返回值一般情况应相同；重写隐藏要求基类的函数不是虚函数或者函数参数不同。
 - 重写覆盖会使派生类虚函数表中基类的虚函数的指针被派生类的虚函数指针覆盖。重写隐藏不会。

重载、重写隐藏与重写覆盖

	重载(overload)	重写隐藏(redefining)	重写覆盖(override)
作用域	相同(同一个类中, 或者均为全局函数)	不同(派生类和基类)	不同(派生类和基类)
函数名	相同	相同	相同
函数参数	不同	相同/不同	相同
返回值	不能仅返回值不同	无要求	相同或协变的
其他要求	—	如果函数参数相同, 则 基类函数不能为虚函数	基类函数为虚函数

重写覆盖同样会隐藏掉基类的同名函数!

重写覆盖

```
#include <iostream>
using namespace std;
class Base{
public:
    virtual void foo(){cout<<"Base::foo()"<<endl;}
    virtual void foo(int ){cout<<"Base::foo(int )"<<endl;} ///重载
    void bar(){};
};
class Derived1 : public Base {
public:
    void foo(int ) {cout<<"Derived1::foo(int )"<<endl;} /// 是重写覆盖
};
class Derived2 : public Base {
public:
    void foo(float ) {cout<<"Derived2::foo(float )"<<endl;} /// 误把参数写错了, 不是
    重写覆盖, 是重写隐藏
};
```

重写覆盖

```
int main() {  
    Derived1 d1;  
    Derived2 d2;  
    Base* p1 = &d1;  
    Base* p2 = &d2;  
    //d1.foo(); ///由于派生类都定义了带参数的foo，基类foo()对实例不可见  
    //d2.foo();  
    p1->foo(); ///但是虚函数表中有继承自基类的foo()虚函数  
    p2->foo();  
    d1.foo(3); ///重写覆盖  
    d2.foo(3.0); ///调用的是派生类foo(float )  
    p1->foo(3); ///重写覆盖，虚函数表中是派生类的 foo(int )  
    p2->foo(3.0); ///重写隐藏，虚函数表中是继承自基类 foo(int )  
    return 0;  
}
```

Base::foo()

Base::foo()

Derived1::foo(int)

Derived2::foo(float)

Derived1::foo(int)

Base::foo(int)

运行结果

Override关键字

- 重写覆盖要满足的条件很多，很容易写错，可以使用**override**关键字辅助检查。
- **override**关键字明确地告诉编译器一个函数是对基类中一个**虚函数**的重写覆盖，编译器将对重写覆盖要满足的条件进行检查，**正确的重写覆盖**才能通过编译。
- 如果没有**override**关键字，但是满足了重写覆盖的各项条件，也能实现重写覆盖。它只是编译器的一个检查，正确实现**override**时，对编译结果没有影响。

Override关键字

```
#include <iostream>
using namespace std;
class Base{
public:
    virtual void foo(){cout<<"Base::foo()"<<endl;}
    virtual void foo(int ){cout<<"Base::foo(int )"<<endl;} ///重载
    void bar(){};
};
class Derived1 : public Base {
public:
    void foo(int ) {cout<<"Derived1::foo(int )"<<endl;} /// 是重写覆盖
};
class Derived2 : public Base {
public:
    void foo(float ) {cout<<"Derived2::foo(float )"<<endl;} /// 误把参数写错了, 不是
    重写覆盖, 是重写隐藏
};
```

Override关键字

```
class Derived3 : public Base {  
public:  
    void foo(int ) override {cout<<"Derived3::foo(int )" <<endl;}; /// 重写覆盖正确,  
与Derived1等价  
    //void foo(float ) override {}; /// 参数不同, 不是重写覆盖, 编译错误  
    //void bar() override {}; /// bar 非虚函数, 编译错误  
};
```

Override关键字

```
int main() {
    Derived1 d1;
    Derived2 d2;
    Derived3 d3;
    Base* p1 = &d1;
    Base* p2 = &d2;
    Base* p3 = &d3;
    //d1.foo(); ///由于派生类都定义了带参数的foo, 基类foo()对实例不可见
    //d2.foo();
    //d3.foo();
    ///但是虚函数表中有继承自基类的foo()虚函数
    p1->foo();
    p2->foo();
    p3->foo();
    d1.foo(3); ///重写覆盖
    d2.foo(3.0); ///调用的是派生类foo(float )
    d3.foo(3); ///重写覆盖
    p1->foo(3); ///重写覆盖
    p2->foo(3.0);///重写隐藏, 调用的是基类foo(int)
    p3->foo(3); ///重写覆盖
    return 0;
}
```

运行结果

```
Base::foo()
Base::foo()
Base::foo()
Derived1::foo(int )
Derived2::foo(float )
Derived3::foo(int )
Derived1::foo(int )
Base::foo(int )
Derived3::foo(int )
```

final关键字

■ 不想让使用者继承? -> final关键字!

- 在**虚函数声明或定义**中使用时，final确保函数为虚且不可被派生类重写。可在继承关系链的“中途”进行设定，禁止后续派生类对指定虚函数重写。
- 在**类定义**中使用时，final指定此类不可被继承。

```
class Base{
    virtual void foo(){};
};
class A: public Base {
    void foo() final {}; /// 重写覆盖，且是最终覆盖
    void bar() final {}; /// bar 非虚函数，编译错误
};
class B final : public A{
    void foo() override {}; /// A::foo 已是最终覆盖，编译错误
};
class C : public B{ /// B 不能被继承，编译错误
};
```

OOP核心思想

- OOP的核心思想是数据抽象、继承与动态绑定
- 数据抽象：类的接口与实现分离
- 继承：建立相关类型的层次关系（基类与派生类）
- 动态绑定：统一使用基类指针，实现多态行为

OOP核心思想

■ OOP的核心思想是数据抽象、继承与动态绑定

■ 数据抽象：类的接口与实现分离

- 纯虚函数、模板（下一讲介绍）

■ 继承：建立相关类型的层次关系（基类与派生类）

- is-a、is-implementing-in-terms-of: 客观世界的认知关系

■ 动态绑定：统一使用基类指针，实现多态行为

- 虚函数、类型转换

课后阅读

- 《C++编程思想》
 - 多态性与虚函数，第15章

重写覆盖的条件（课后探究）

If some member function `vf` is declared as `virtual` in a class `Base`, and some class `Derived`, which is derived, directly or indirectly, from `Base`, has a declaration for member function with the same

- name
- parameter type list (but not the return type)
- cv-qualifiers
- ref-qualifiers

Then this function in the class `Derived` is also *virtual* (whether or not the keyword `virtual` is used in its declaration) and *overrides* `Base::vf` (whether or not the word `override` is used in its declaration).

`Base::vf` does not need to be visible (can be declared private, or inherited using private inheritance) to be overridden.

■ 一个例子：

- 使用`const`修饰成员函数，可能导致重写覆盖失效

进一步阅读： <https://en.cppreference.com/w/cpp/language/virtual>

```
#include <iostream>
using namespace std;
```

```
class Base1{
public:
    virtual void f()
        {cout << "Base1::f" << endl;}
};
```

```
class Derive1: public Base1{
public:
    //重写覆盖失效，其实是重写隐藏
    void f() const
        {cout << "Derive1::f" << endl;}
    //使用using恢复被隐藏的基类函数
    using Base1::f;
};
```

```
class Base2{
public:
    virtual void g()
        {cout << "Base2::g" << endl;}
};
class Derive2:public Base2{
public:
    void g() //重写覆盖
        {cout << "Derive2::g" << endl;}
    using Base2::g;
};
```

const对重写覆盖和重写隐藏的影响

```
int main(){
    Derive1 a;
    const Derive1 b;
    a.f(); //Base1::f已被恢复，非常量对象优先
           匹配Base1::f
    b.f(); //常量对象调用Derive1::f

    Base2 c;
    Derive2 d;
    c.g();
    d.g(); //重写覆盖，调用Derive2::g
    return 0;
}
```

运行结果

```
Base1::f
Derive1::f
Base2::g
Derive2::g
```

虚函数的返回值（课后探究）

- 一般来说，派生类虚函数的返回类型应该和基类相同；
- 或者，是协变(Covariant)的，例如
 - 基类和派生类的指针是协变的
 - 基类和派生类的引用是协变的

```
#include <iostream>
using namespace std;
```

```
class Instrument {
public:
    virtual Instrument& getObj() { return *this; }
};
```

```
class Wind : public Instrument {
public:
    virtual Wind& getObj() { return *this;}
    //Wind&和Instrument&协变
};
```

去掉引用是否能够编译？

虚函数的返回值类型（课后探究）

```
class Base {  
public:  
    virtual ReturnType1 f(argument){}  
};  
  
class Derive : public Base {  
    ReturnType2 f(argument){}  
};
```

■ 虚函数的返回类型需要满足如下两个条件之一：

- Derive::f返回类型与Base::f的相同
- Derive::f返回类型和Base::f的返回类型是协变的，即满足如下所有条件：
 - 都是指针（不能是多级指针）、都是左值引用或都是右值引用，且在Derive::f声明时，Derive::f的返回类型必须是Derive或其他已经完整定义的类型
 - ReturnType1中被引用或指向的类是ReturnType2中被引用或指向的类的祖先类
 - Base::f的返回类型相比Derive::f的返回类型同等或更加[cv-qualified](https://en.cppreference.com/w/cpp/language/virtual)

进一步阅读： <https://en.cppreference.com/w/cpp/language/virtual>

虚函数返回类型

```
class A{};
class B : public A{};
class C{};
class D : public B, public C{};
```

```
class Base {
public:
    virtual Base* f1(){}
    virtual Base** f2(){}
    virtual Base& f3(){}
    virtual A& f4(){}
};
```

```
class Derive : public Base {
public:
```

```
    Derive* f1(){} //返回值类型都是指针，Base是Derive的祖先类
```

```
    //Derive** f2(){} //编译错误，不能是多级指针
```

```
    Base** f2(){} //返回值类型相同
```

```
    Derive& f3(){} //返回值类型都是引用，Base是Derive的祖先类
```

```
    //Derive* f3(){} //编译错误，类型不同，且非协变
```

```
    D& f4(){} //返回值类型都是引用，A是D的祖先类
```

```
};
```

使用虚函数实现多态（课后练习）

- 根据以下代码实现 **Animal**、**Bird**、**Fish** 类

```
void action(Animal* pAnimal) {  
    pAnimal -> sing();  
    pAnimal -> swim();  
}  
  
int main(){  
    Animal *myBird = new Bird();  
    Animal *myFish = new Fish();  
    action(myBird);  
    action(myFish);  
    delete myBird;  
    delete myFish;  
    return 0;  
}
```

参考输出：

```
bird is singing.  
bird can't swim.  
fish can't sing.  
fish is swimming.  
bird has gone.  
fish has gone.
```

结束

利用返回值优化提高执行效率

```
#include <iostream>
using namespace std;

class Test{
public:
    int data = 0;
    Test(){}
    Test(const Test& t){}
    Test(Test&& t){}
};

Test fn1(){
    Test tmp; return tmp; //(1)
}
Test&& fn2(){
    Test tmp; return move(tmp); //(2)
}
Test fn3(){
    Test tmp; return move(tmp); //(3)
}

int main(){
    const Test& a = fn1(); //(4)
    Test&& b = fn1(); //(5)
    Test c = fn1(); //(6)
    Test&& d = fn2(); //(7)
    return 0;
}
```

■ 返回值优化条件

- 返回的值类型与函数签名的返回值类型相同
- 返回的是一个局部对象的左值

■ 建议做法包括(1)(4)(5)(6),避免多余拷贝,优化资源利用

- Test fn1(); 满足返回值优化条件
- 可利用常量左值引用 (4) , 右值引用 (5) , 构造新对象 (6) 的方式接收返回值

■ 不建议做法包括(2)(3)(7)

- (2)(7) d会指向被析构的tmp, 出现运行错误
- std::move()将左值转变为右值, 不进行返回值优化, (3)会移动构造临时变量